PROJEKTDOKUMENTATION

Elektronischer Synthesizer

auf Basis des ESP32-S3 Mikrocontrollers

Bearbeiter:

Luis Koch

Betreuer:
Herr Spitzer

Inhaltsverzeichnis

Inhaltsverzeichnis....2
1. Einleitung....4
2. Planung / GANT Diagramm....4
2.1 Projektziele....4
2.2 Hardware-Auswahl....5
3. Grundlagen....5
3.1 ESP32 / ESP32-S3....5
3.2 I2S (Inter-IC Sound)....5
3.3 MAX98357A....6
3.3.1 Funktionsprinzip....6
3.3.2 Elektrische Eigenschaften....7
3.4 Digitale Tonerzeugung....7
3.5 Kapazitive Touch-Eingänge....7
4. Realisierung....7
4.1 Hardware....7
4.1.1 Audio-Schaltung....7
4.1.2 Touch-Pads....7
4.1.2 Schaltplan....8
4.3 Fräsen der Kupfer-Touchpads....8
4.3.1 Vorbereitung....8
4.3.2 Frässtrategien....8
4.3.3 Ergebnis....9
4.3 Gehäuse – 3D-Druck....9
4.3.1 Konstruktion....9
4.3.2 Druckparameter (privater Drucker)....9
4.3.3 3D-Design....9
4.3.4 Montage....11
4.4 Testfunktionen....11
4.4.1 Audio-Test....11
4.4.2 Touch-Test....11
4.5 Gesamtfunktion....11
4.6 Polyphonie....11
4.6.1 Individuelle Phasenverfolgung....11
4.6.2 Sample-Mischung im 32-Bit-Puffer....12
4.6.3 Normalisierung und Clipping-Schutz....12
4.7 ADSR-Hüllkurve....12
4.7.1 Attack....12
4.7.2 Decay....13
4.7.3 Sustain....13
4.7.4 Release....13
4.7.5 Zusammenspiel mit der Normalisierung....14
5. Inbetriebnahme und Test....14

6. Fehlersuche, Beseitigung und Verbesserung 14
6.1 Probleme 14
6.2 Ursachen 14
6.3 Lösungen 15
7. Ergebnisse 15
8. Fazit 15
8.1 Vergleich Planung – Realität 15
8.2 Verbesserungsmöglichkeiten 15
8.3 Neue Ideen 15
9. Anhang 16
9.1 Datenblätter 16
9.2 API-Beschreibungen 16
9.3 Anleitungen 16
9.6 KI-Unterstützung und verwendete Prompts 16
Google Gemini – Interferenzberechnung 16
ChatGPT - Allgemeine Unterstützung 17
Claude AI (Anthropic) 17
10. Quellen 18
Bilder 18
11. Englisches Summary 19
Introduction 19
Main Part 19
Conclusion 19

Erklärung

Die vorliegende Projektarbeit habe ich eigenständig und ausschließlich mit den angegebenen Hilfsmitteln verfasst.

Alle Stellen, die sinngemäß oder wörtlich aus anderen Werken übernommen wurden, sind durch entsprechende Quellenangaben kenntlich gemacht. Wörtliche Übernahmen sind zudem als Zitate gekennzeichnet.

Mannheim, den _____ Unterschrift _____

1. Einleitung

Ziel dieses Projekts ist die Entwicklung eines elektronischen Musikinstruments auf Basis eines ESP32-Mikrocontrollers. Der ESP32 erzeugt digitale Audiosignale und gibt diese über einen externen Audioverstärker wieder. Die Tonhöhe wird über kapazitive Touch-Eingänge gesteuert, die aus gefrästen Kupferflächen bestehen.

Das Konzept orientiert sich an Instrumenten wie dem Suzuki Omnichord oder einem einfachen Synthesizer.

Mögliche Methoden zur Tonerzeugung mit dem ESP32:

Methode	Gleichzeitige Töne
Hardware-PWM	bis zu 16 Töne (abhängig von PWM-Kanälen)
DAC (einfach)	1–2 Töne (Software-Mischung möglich)
DAC + Software-Mischung	4–8 Töne möglich
I2S + externes Modul	so viele, wie das Modul unterstützt

Für dieses Projekt wurde I2S in Kombination mit einem ESP32-S3 und dem MAX98357A-Verstärker gewählt, da dieser Ansatz die beste Audioqualität bietet.

2. Planung / GANT Diagramm

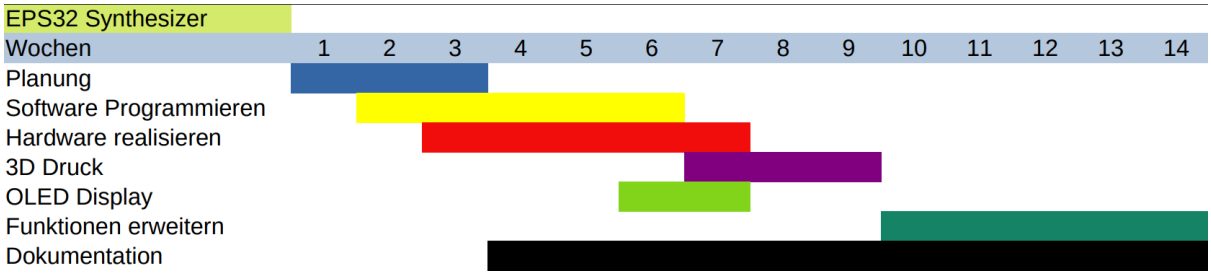

Schaubild 1

2.1 Projektziele

- Erzeugung von hörbaren Tönen mit einstellbarer Frequenz
- Nutzung von I2S für qualitativ hochwertige Audioausgabe
- Steuerung der Tonhöhe über Touch-Eingänge
- Möglichkeit, Noten und Melodien zu programmieren
- Stabiler Betrieb ohne Abstürze oder Audiostörungen

2.2 Hardware-Auswahl

- ESP32-S3 Mikrocontroller
- MAX98357A I2S-Audioverstärker
- Lautsprecher
- Gefräste Kupfer-Touchpads
- Taster zur Zusatzsteuerung

3. Grundlagen

3.1 ESP32 / ESP32-S3

Der ESP32-S3 ist ein leistungsfähiger Mikrocontroller mit folgenden relevanten Eigenschaften:

- Dual-Core CPU
- Integrierte I2S-Schnittstelle für digitale Audioübertragung
- Kapazitive Touch-Eingänge (hardwareseitig)
- Keine internen DACs (im Gegensatz zum klassischen ESP32)
- Veränderte Touch-Hardware gegenüber dem Vorgängermodell
- Stärkere USB-Integration

3.2 I2S (Inter-IC Sound)

I2S (Inter-IC Sound) ist ein serielles Protokoll zur Übertragung digitaler Audiodaten zwischen Bausteinen wie Mikrocontrollern, DACs oder ADCs. Es ist auf kontinuierliche, zeitkritische Audiosignale ausgelegt.
Es nutzt drei Leitungen:

- BCLK (Bit-Clock): Takt für jedes Bit
- LRCK (Word-Select): Unterscheidet linken und rechten Kanal
- DIN (Datenleitung): Überträgt die Audiodaten (MSB zuerst)

Die Daten werden kanalweise (Stereo) übertragen und sind exakt mit den Taktsignalen synchronisiert. Ein Gerät gibt als Master den Takt vor, andere arbeiten als Slave.

Typisch sind 16–32 Bit pro Sample und Samplingraten wie 44,1 kHz oder 48 kHz.

3.3 MAX98357A

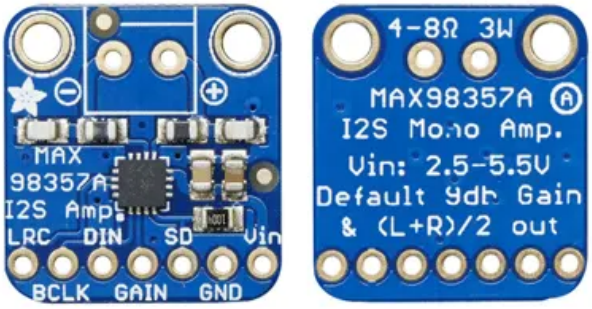

Schaubild 2

Der MAX98357A ist ein digitaler Class D-Audioverstärker des Herstellers Analog Devices (ehemals Maxim Integrated), der speziell für den Einsatz in kompakten, kostengünstigen Audioprojekten entwickelt wurde. Er vereint einen vollständigen Digital-Analog-Wandler (DAC) und eine Verstärkerstufe in einem einzigen Chip und kann direkt mit einem Lautsprecher verbunden werden – ohne externe Filter oder zusätzliche Bauteile.

3.3.1 Funktionsprinzip

Der Chip empfängt digitale Audiodaten über das I2S-Protokoll. Dabei werden drei Leitungen benötigt: BCLK (Bit-Clock), LRCLK (Left-Right-Clock, auch Word-Select genannt) und DIN (Datenleitung). Der interne DAC wandelt die eingehenden 16-Bit-Digitaldaten in ein analoges Signal um, das anschließend von der Class-D-Endstufe direkt auf den Lautsprecher gegeben wird. Class-D bedeutet, dass die Ausgangsstufe nicht linear arbeitet, sondern mit einer hochfrequenten Schaltfrequenz von 300 kHz schaltet. Die Induktivität der Lautsprecherspule wirkt dabei als natürlicher Tiefpassfilter und rekonstruiert das Audiosignal. Dadurch entfällt die Notwendigkeit eines externen LC-Filters, was den Schaltungsaufwand erheblich reduziert.

Ein besonders wichtiges Merkmal für dieses Projekt ist, dass der MAX98357A kein externes MCLK-Signal benötigt. Dieses Signal ist bei vielen anderen I2S-Geräten zwingend erforderlich und muss vom Mikrocontroller erzeugt werden. Der MAX98357A erkennt die Taktrate automatisch anhand von BCLK und LRCLK, was die Anbindung an den ESP32-S3 deutlich vereinfacht und mögliche Taktprobleme von vornherein ausschließt.

3.3.2 Elektrische Eigenschaften

Der Chip wird mit einer Versorgungsspannung zwischen 2,5 V und 5,5 V betrieben, was ihn sowohl für 3,3-V- als auch für 5-V-Systeme geeignet macht. Bei 5 V Versorgungsspannung liefert er eine Ausgangsleistung von bis zu 3,2 W an einem 4-Ohm-Lautsprecher. Der Wirkungsgrad liegt laut Datenblatt bei bis zu 92 %, was bedeutet dass nur ein kleiner Anteil der aufgenommenen Energie als Wärme verloren geht. Der Ruhestrom beträgt lediglich 2,4 mA, was den Chip auch für batteriebetriebene Anwendungen interessant macht.

Die Verstärkung lässt sich über den Pin GAIN_SLOT hardwareseitig in fünf Stufen einstellen: 3 dB, 6 dB, 9 dB, 12 dB oder 15 dB. Dies geschieht durch einfaches Verbinden des Pins mit VDD, GND oder einem definierten Widerstand – ohne Software-Konfiguration. Für dieses Projekt wurde die Verstärkung auf 9 dB eingestellt, indem der Pin unbeschaltet gelassen wurde.

Die unterstützten Sampleraten reichen laut Datenblatt von 8 kHz bis 96 kHz, was für das vorliegende Projekt mit einer Samplerate von 16 kHz problemlos ausreicht. Das Audioformat ist auf 16-Bit-Daten im I2S-Standard ausgelegt, was genau dem entspricht was der ESP32-S3 über seinen I2S-Treiber ausgibt.

3.4 Digitale Tonerzeugung

Die Tonerzeugung erfolgt durch mathematische Berechnung von Wellenformen (z. B. Sinus) und deren periodische Ausgabe über I2S. Die Tonhöhe wird über die Schrittweite der Phase bestimmt.

3.5 Kapazitive Touch-Eingänge

Touch-Pins messen die Kapazität eines angeschlossenen Pads. Berührt ein Finger das Pad, erhöht sich die Kapazität messbar, was als Touch-Ereignis ausgewertet wird.

4. Realisierung

4.1 Hardware

4.1.1 Audio-Schaltung

- ESP32-S3 → MAX98357A (I2S-Verbindung)
- MAX98357A → Lautsprecher
- Versorgung über 5 V und GND

4.1.2 Touch-Pads

- Kupferflächen direkt mit Touch-GPIOs verbunden

4.1.2 Schaltplan

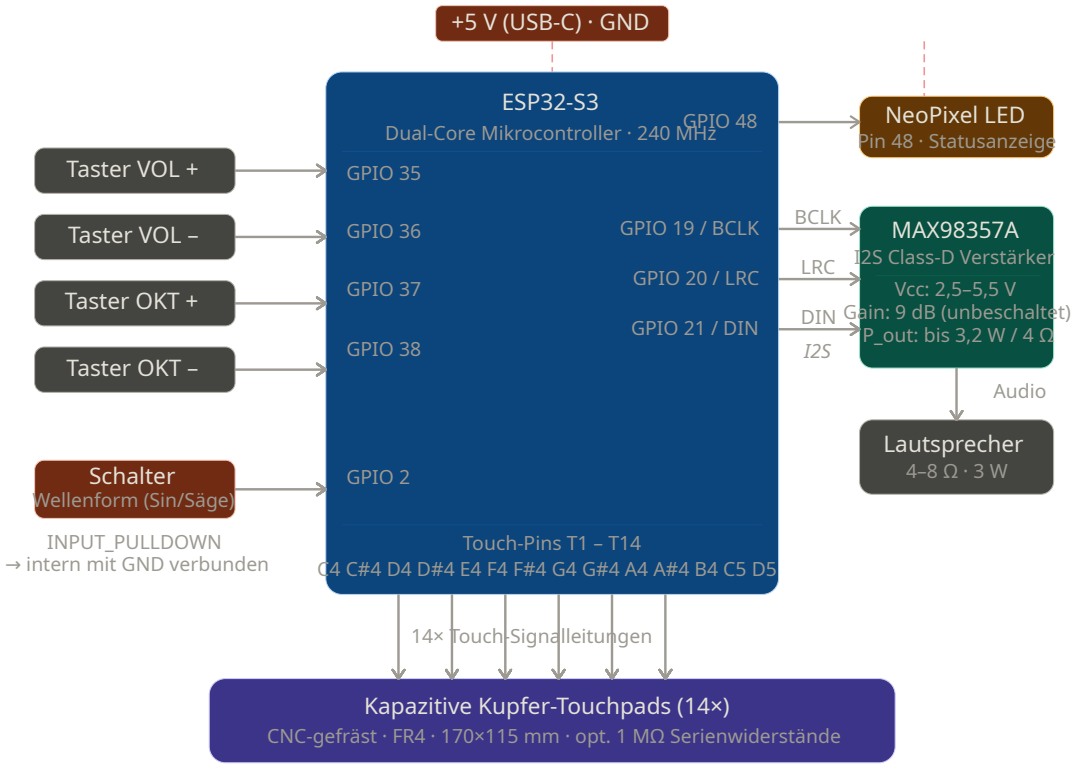

Schaubild 3

4.2 Fräsen der Kupfer-Touchpads

Die Kupfer-Touchpads wurden durch Fräsen auf einer CNC-Fräsmaschine aus einer einseitigen Kupferplatine hergestellt. Herr Fech hat angeboten, dies zu übernehmen, sofern die genauen Abmessungen bereitgestellt werden.

4.2.1 Vorbereitung

- Material: einseitige Kupferplatine (FR4)
- Skizze für die Fräsung auf Papier gezeichnet
- Größe: 170 mm × 115 mm
- Platinendicke: 2 mm

4.2.2 Frässtrategien

- Isolationsfräsung der Leiterbahnen
- Konturschnitt zum Ausfräsen der Platinenform

4.2.3 Ergebnis

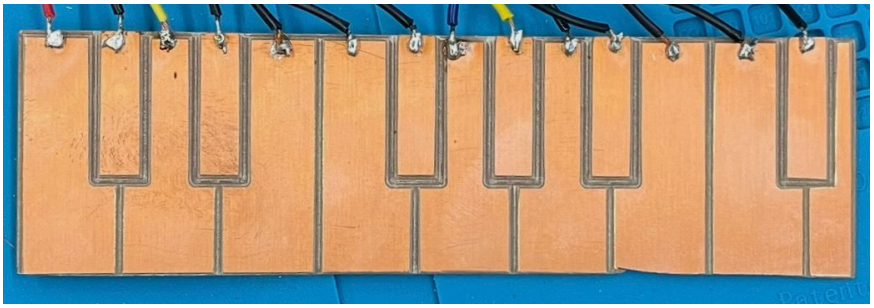

Schaubild 4

4.3 Gehäuse – 3D-Druck

Das Gehäuse wurde mithilfe eines 3D-Druckers gefertigt. Es dient als stabile Halterung für die Elektronik und macht die Touchpads ergonomisch zugänglich. Genutzt wurden ein Drucker aus dem MakerSpace (Creality K2 Plus) sowie ein privater Ender 3.

4.3.1 Konstruktion

- Konstruktionssoftware: Autodesk TinkerCAD
- Exportformat: STL
- Abmessungen: 141 mm × 190 mm × 45 mm

4.3.2 Druckparameter (privater Drucker)

Für den privaten Ender-3-Drucker wurden Einstellungen für niedrigere Qualität gewählt, um die Druckzeit zu reduzieren:

- 3D-Drucker: Ender 3
- Material: PLA
- Schichthöhe: 0,28 mm
- Fülldichte (Infill): 10 %
- Druckgeschwindigkeit: 70 mm/s

4.3.3 3D-Design

Nachdem zunächst eine vollständige Box geplant war, wurde in Absprache mit Herrn Spitzer eine ressourcenschonendere Lösung gefunden: Ein bereits vorhandenes Testmodul konnte so umgenutzt werden, dass nur die Frontplatte komplett neu konstruiert und gedruckt werden musste. Das spart Material und Druckzeit, ohne die Funktionalität einzuschränken.

Zusätzlich zur Frontplatte wurden separate Halterungen für die Oktaven- und

Lautstärketaster gedruckt. Diese wurden so konstruiert, dass die Taster fest sitzen und beim Drücken nicht nachgeben oder verrutschen. Auch die Kupfer-Touchpads bekamen eigene Halterungen.

Für die Faceplate habe ich das recycelte Testmodul abgemessen, die Maße betragen: 160mm x 135mm x 3mm.

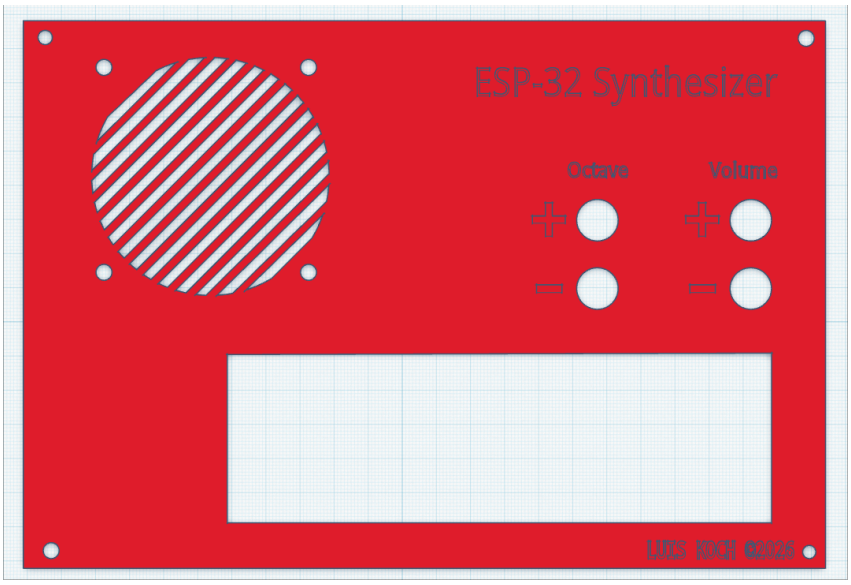

Schaubild 5

Bei der Halterung der Taster betragen die Maße: 49mm x 72mm x 49mm. Und für die der Kupfer-Touchpads sind es: 55mm x 180mm x 5mm.

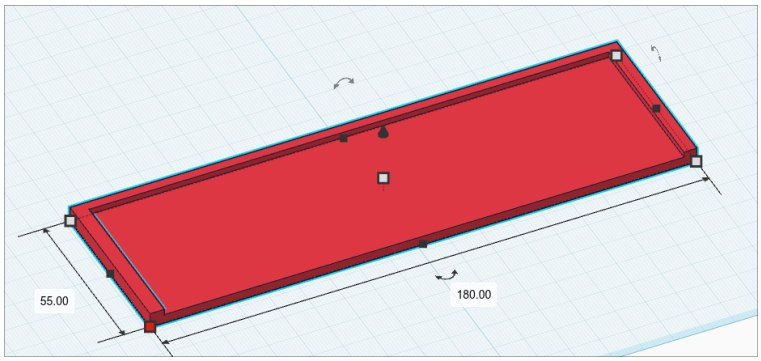

Schaubild 7

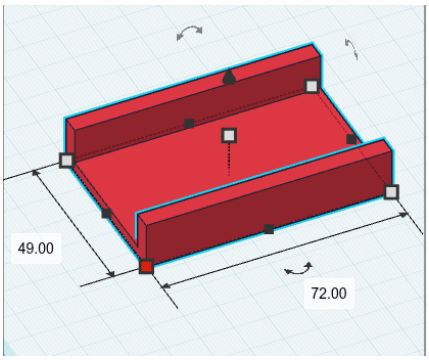

Schaubild 6

4.3.4 Montage

Nach mehreren Anpassungen der Lochmaße passen alle Komponenten. Die Lautstärke- und Oktaven-Taster wurden auf eine Lochrasterplatine gelötet. Halterungen für den ESP32, die Tasterplatine und die Kupferpads werden noch modelliert und gedruckt; sie werden voraussichtlich mit Kleber an der Frontplatte befestigt.

4.4 Testfunktionen

4.4.1 Audio-Test

- Ausgabe eines festen Sinustons (440 Hz / Kammerton A)
- Prüfung der Audioqualität und Lautstärke

Der Test war erfolgreich; die Audioqualität ist sehr gut.

4.4.2 Touch-Test

- Ausgabe der Rohwerte über den seriellen Monitor
- Bestimmung geeigneter Schwellwerte

Nach mehreren Tests wurden folgende Schwellwerte ermittelt: thresholds[] = {6000, 6000, 6000, 6000, 6000, 6000, 600, 6000}. Diese funktionieren für die Testversion der Kupferpads; nach Fertigstellung der finalen Kupferpads werden die Werte entsprechend angepasst.

4.5 Gesamtfunktion

- Touch-Pads werden zyklisch abgefragt
- Jede Taste ist einer Frequenz zugeordnet
- Die aktuell erkannte Note bestimmt die Tonhöhe
- Audio wird kontinuierlich über I2S ausgegeben

4.6 Polyphonie

Polyphonie bedeutet, dass mehrere Töne gleichzeitig gespielt werden können. Im Code wird dies durch drei zusammenwirkende Mechanismen realisiert: individuelle Phasenverfolgung je Taste, Sample-Mischung in einem 32-Bit-Puffer und dynamische Lautstärkenormalisierung.

4.6.1 Individuelle Phasenverfolgung

Jede der 14 Tasten besitzt eine eigene Phase im Array phases[NUM_KEYS]. Die Phase gibt an, an welcher Stelle der Wellenformtabelle sich eine Taste gerade befindet. Pro Audio-Sample wird die Phase jeder aktiven Taste um eine frequenzabhängige Schrittweite erhöht. Da jede Taste ihre eigene Phase unabhängig

weiterführt, klingen alle gleichzeitig gedrückten Tasten durch und beeinflussen sich gegenseitig nicht in ihrer Tonhöhe oder ihrem Rhythmus.

4.6.2 Sample-Mischung im 32-Bit-Puffer

Für jedes der 512 Samples pro Durchlauf werden die Einzelwerte aller aktiven Tasten zu einer Variable mixedSample addiert. Diese Variable ist bewusst als int32_t (32 Bit) deklariert, damit beim Addieren mehrerer 16-Bit-Werte kein Überlauf auftritt. Das Ergebnis entspricht der Überlagerung aller gleichzeitig gespielten Wellenformen.

4.6.3 Normalisierung und Clipping-Schutz

Sind mehrere Tasten aktiv, wird der gemischte Wert durch activeKeys × 0,8 geteilt. Der Faktor 0,8 statt 1,0 sorgt dafür, dass der Gesamtklang etwas füller klingt als bei einer exakten Aufteilung. Abschließend wird das Ergebnis mit constrain() hart auf den gültigen 16-Bit-Bereich begrenzt, um Verzerrungen durch Übersteuerung zu verhindern.

4.7 ADSR-Hüllkurve

Um einen natürlicheren und musikalisch ansprechenderen Klang zu erzeugen, wurde eine vollständige ADSR-Hüllkurve implementiert. ADSR steht für Attack, Decay, Sustain und Release und beschreibt die vier Phasen, die die Lautstärke eines Tons über seine gesamte Lebensdauer hinweg steuern. Dieses Prinzip ist ein Grundbestandteil nahezu jedes analogen und digitalen Synthesizers und findet sich in Instrumenten wie dem Moog, dem Roland Juno oder modernen Software-Synthesizern.

Die Umsetzung erfolgt über einen Zustandsautomat. Jede der 14 Tasten besitzt einen eigenen Zustand im Array `adsrState[NUM_KEYS]`, der einen der fünf möglichen Werte annehmen kann: `IDLE`, `ATTACK`, `DECAY`, `SUSTAIN` oder `RELEASE`. Pro berechnetem Sample wird für jede Taste geprüft, in welchem Zustand sie sich befindet, und die Amplitude entsprechend angepasst. Da alle Tasten vollständig unabhängig voneinander laufen, ist volle Polyphonie gewährleistet, jede Taste kann sich gleichzeitig in einer anderen ADSR-Phase befinden.

4.7.1 Attack

Sobald ein Touch-Pad berührt wird und der Messwert den Schwellenwert überschreitet, wechselt die zugehörige Taste in den Zustand `ATTACK`. In diesem Zustand wird die Amplitude pro Sample um den Wert `ATTACK_RATE = 0.1` linear erhöht:

```
amplitude[k] += ATTACK_RATE;
```

Da pro Loop-Durchlauf 512 Samples berechnet werden und die Samplerate 16.000 Hz beträgt, dauert ein einzelner Loop-Durchlauf etwa 32 Millisekunden. Mit einem Attack-Rate von 0.1 wird die Amplitude von 0.0 auf 1.0 in etwa 10 Schritten angehoben, was einer Einschwingzeit von wenigen Millisekunden entspricht. Das klingt sehr direkt und perkussiv, ähnlich einem Klavieranschlag. Ein kleinerer Wert wie 0.001 würde ein weiches, langsames Anschwellen erzeugen, wie es etwa bei Streichinstrumenten vorkommt. Erreicht die Amplitude den Wert 1.0, wechselt die Taste automatisch in die nächste Phase.

4.7.2 Decay

Im Zustand `DECAY` fällt die Amplitude nach dem initialen Peak langsam ab, während die Taste noch gedrückt gehalten wird. Dies geschieht durch multiplikative Dämpfung pro Sample:

```
amplitude[k] *= DECAY_RATE;
```

Mit `DECAY_RATE = 0.9995` wird die Amplitude pro Sample auf 99,95% des vorherigen Wertes gesetzt. Da dies exponentiell geschieht, also bei jedem Schritt prozentual vom aktuellen Wert abgezogen wird, klingt der Abfall für das menschliche Ohr gleichmäßig und natürlich. Die Phase endet sobald die Amplitude das Sustain-Level erreicht hat, woraufhin automatisch in den nächsten Zustand gewechselt wird.

4.7.3 Sustain

Der Sustain-Zustand ist die stabilste Phase der Hüllkurve. Solange die Taste gedrückt gehalten wird und das Sustain-Level erreicht ist, bleibt die Amplitude konstant auf dem Wert `SUSTAIN_LEVEL = 0.8`.

```
amplitude[k] = SUSTAIN_LEVEL;
```

Das bedeutet der Ton wird auf 80% der maximalen Lautstärke gehalten. Dieser Wert wurde bewusst unterhalb von 1.0 gewählt, um den initialen Anschlag (das kurze Aufleuchten auf voller Lautstärke im Attack) klanglich wahrnehmbar zu machen. Würde Sustain auf 1.0 gesetzt, wäre kein Unterschied zwischen Anschlag und gehaltenem Ton hörbar. Der Sustain-Zustand bleibt so lange aktiv bis der Finger das Touch-Pad verlässt.

4.7.4 Release

Sobald der Messwert eines Touch-Pads unter den Schwellenwert fällt, wechselt die Taste in den Zustand `RELEASE`. Ab diesem Punkt läuft der Ton weiter, auch ohne dass die Taste noch berührt wird, und klingt exponentiell aus:

```
amplitude[k] *= RELEASE_RATE;
```

Mit `RELEASE_RATE = 0.9990` beträgt die Ausklingzeit etwa eine halbe Sekunde.

Dieser Wert wurde gewählt um ein natürliches Nachklingen zu erzeugen, ohne dass die Töne zu lange im Raum stehen bleiben und die Spielbarkeit beeinträchtigen. Fällt die Amplitude unter den Schwellenwert 0.001, wechselt die Taste in den Zustand `IDLE` und die gesamte Audioberechnung für diese Taste wird eingestellt. Das spart Rechenzeit und verhindert, dass unhörbare Minimalwerte dauerhaft in den Mix einfließen.

4.7.5 Zusammenspiel mit der Normalisierung

Da mehrere Tasten gleichzeitig aktiv sein können und sich alle in unterschiedlichen ADSR-Phasen befinden, variiert die Gesamtlautstärke des Mixes ständig. Um Übersteuerung zu verhindern, wird die Summe aller aktiven Amplituden `ampSum` berechnet und der gemischte Sample entsprechend normalisiert. So bleibt der Ausgangspegel auch beim gleichzeitigen Anschlagen mehrerer Tasten stabil, ohne zu verzerren.

5. Inbetriebnahme und Test

- Überprüfung der seriellen Ausgabe
- Test der Audioausgabe
- Test der ADSR-Hüllkurve
- Test einzelner und mehrerer Touch-Eingaben
- Anpassung von Schwellenwerten und Lautstärke

Die Ausgabe mit Polyphonie funktioniert zum Teil; mehrere Noten sind spielbar. Folgende Probleme sind dabei aufgetreten:

- Serielle Ausgabe friert bei gleichzeitigem Drücken mehrerer Pads ein
- Touch-Werte ändern sich bei mehreren gleichzeitigen Berührungen kaum

6. Fehlersuche, Beseitigung und Verbesserung

6.1 Probleme

- Serielle Ausgabe friert ein – bei vielen gleichzeitigen Noten stärker als bei wenigen
- Touch-Werte ändern sich bei mehreren gleichzeitigen Berührungen kaum
- Knacken beim Frequenzwechsel
- Bootloops bei falscher I2S-Konfiguration

6.2 Ursachen

- Touch-Hardware-Grenzen des ESP32-S3: Die S3-Version misst Touch-Werte

anders als das Originalmodell – sie zählt Zyklen in einem festen Zeitraum, während das Original die Zeit bis zum Erreichen einer RC-Spannung misst.

- Zu häufiges Aufrufen von touchRead()
- Zurücksetzen der Phase beim Frequenzwechsel

6.3 Lösungen

- Touch nur alle 30–50 ms abfragen
- Schwellenwert-Logik statt exakter Vergleichswerte
- Phase kontinuierlich weiterlaufen lassen
- Geeignete Touch-fähige GPIO-Pins verwenden

Das Knacken beim Frequenzwechsel und die Bootloops wurden dadurch behoben. Das Einfrieren der seriellen Ausgabe konnte nicht vollständig gelöst werden, da es die Hardware-Grenzen des ESP32-S3 überschreitet. Die einzige grundlegende Lösung wäre der Wechsel auf ein anderes Mikrocontroller-Modell.

7. Ergebnisse

- Erfolgreiche Tonerzeugung über I2S
- Spielbare Touch-Tastatur mit mehreren Noten
- Programmierbare Melodien
- Stabiler Betrieb bei angepasster Touch-Abfrage

8. Fazit

8.1 Vergleich Planung – Realität

- Grundkonzept erfolgreich umgesetzt
- Touch-Eingänge gestalteten sich schwieriger als geplant
- Audioqualität sehr gut dank I2S
- 3D-Druck zeitaufwendiger als geplant

8.2 Verbesserungsmöglichkeiten

- Trennung von Audio- und Touch-Tasks (Dual-Core-Nutzung)
- ADSR Werte anpassen
- Weitere Wellenformen: Sägezahn, Rechteck

9. Anhang

https://github.com/kochluis598-a11y/ESP32-Synth.git

9.1 Datenblätter

- ESP32-S3 Technical Reference Manual
- MAX98357A Datasheet (Analog Devices)

9.2 API-Beschreibungen

- ESP32 I2S Driver (driver/i2s.h)
- Arduino touchRead()

9.3 Anleitungen

- Arduino ESP32 Core Dokumentation
- I2S Audio auf ESP32

9.4 KI-Unterstützung und verwendete Prompts

Im Rahmen dieses Projekts wurden KI-Werkzeuge zur Unterstützung bei der Fehlersuche sowie bei der Berechnung und Erklärung von Konzepten eingesetzt. In der GitHub-Repo sind die Chatverläufe mit den folgenden KI Modellen dokumentiert.

Google Gemini – Interferenzberechnung

- Prompt 1: (Quellcode angehängt) Wie kann ich möglichst effektiv hier Polyphonie erreichen?
- Prompt 2: Ich addiere bei meinem ESP32-Projekt mehrere Sinuswellen um Polyphonie zu erreichen. Wenn ich 3 Tasten gleichzeitig drücke wird der Ton viel zu laut und verzerrt. Wie berechne ich die Interferenz richtig und wie normalisiere ich den gemischten Audiopegel damit kein Clipping entsteht?
- Prompt 3: Was ist der Unterschied zwischen konstruktiver und destruktiver Interferenz bei Überlagerung von Sinuswellen? Und warum klingt die Summe von zwei gleichfrequenten Wellen doppelt so laut statt gleich laut?
- Prompt 4: Erklär mir wie ich in C++ einen int32_t als Mix-Puffer benutze um 16-Bit Audio-Samples sicher zu addieren ohne Überlauf, und danach wieder auf int16_t begrenze.

ChatGPT - Allgemeine Unterstützung

- Prompt: Ist mein Projekt realisierbar: Ich will mit einem ESP32-S3 und dem MAX98357A Verstärker über I2S Sinustöne erzeugen und die Tonhöhe über

kapazitive Touch-Pads steuern. Was muss ich dabei beachten?

- Prompt: Mein ESP32-S3 crasht beim Starten sobald ich I2S initialisiere. Die serielle Ausgabe hört einfach auf. Was können Gründe dafür sein und wie finde ich den Fehler?
- Prompt: Meine Touch-Werte mit touchRead() auf dem ESP32-S3 sind sehr unzuverlässig und ändern sich kaum wenn ich mehrere Finger gleichzeitig benutze. Woran liegt das und wie kann ich das verbessern?
- Prompt: Wie berechne ich aus einer gewünschten Frequenz in Hz die richtige Schrittweite um eine Wellenform-Lookup-Tabelle mit 256 Einträgen bei einer Samplerate von 16000 Hz korrekt abzutasten?
- Prompt: Ich will auf dem ESP32-S3 einen Neopixel LED als visuelles Feedback nutzen. Wie initialisiere ich die Adafruit NeoPixel Bibliothek und setze die Farbe abhängig davon ob gerade eine Taste gedrückt wird?

Claude AI (Anthropic Sonnet 4.6)

- Prompt: (Bewertungskriterien für die Dokumentation angehängt) Mach mir eine Struktur für die Dokumentation im .odt Dateiformat für LibreOffice.
- Prompt: (Dokumentation angehängt) Korrigiere und verbessere schlechten Satzbau und Schreibfehler.
- Prompt: (Quellcode angehängt) Zeichne anhand von diesem Code einen Schaltplan

10. Quellen

- Espressif Systems: ESP32-S3 Technical Reference Manual – https://docs.espressif.com/projects/esp-idf/en/latest/esp32s3/ (abgerufen: 10.11.24)
- Espressif Systems: ESP32 I2S API Reference – https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/peripherals/i2s.html (abgerufen: 15.11.24)
- Analog Devices / Maxim Integrated: MAX98357A Datasheet – https://www.makerguides.com/wp-content/uploads/2026/01/Datasheet_MAX98357A.pdf (abgerufen: 03.12.24)
- Arduino ESP32 Core GitHub – https://github.com/espressif/arduino-esp32 (abgerufen: 10.11.24)
- Eigene Messungen und Tests

10.1 Bilder

- Schaubild 1: Mit privatem Mobiltelefon aufgenommen
- Schaubild 2: LibreOffice Calc
- Schaubild 3: https://www.makerguides.com/playing-audio-with-esp32-and-max98357/ (abgerufen: 20.04.26)
- Schaubild 4: Generiert mit Claude (Anthropic) https://claude.ai/
- Schaubild 5: Mit privatem Mobiltelefon aufgenommen
- Schaubild 6: https://www.tinkercad.com/ (abgerufen: 24.04.26)
- Schaubild 7: https://www.tinkercad.com/ (abgerufen: 24.04.26)
- Schaubild 8: https://www.tinkercad.com/ (abgerufen: 24.04.26)

11. Englisches Summary

Introduction

The goal of this project was to develop a functional electronic musical instrument using an ESP32-S3 microcontroller. The idea was to create a synthesizer that can be played by touching copper sensor surfaces, which detect human contact through changes in capacitance. The generated tones are calculated digitally and transmitted via an I2S audio interface to an amplifier, which drives a connected speaker. The project combines multiple fields, including microcontroller programming, digital signal processing, and electronics. The aim was to build a working prototype capable of playing multiple notes simultaneously, producing different pitches, and offering basic controls such as volume and octave adjustment.

Main Part

The hardware setup consists of an ESP32-S3 as the central processing unit, a MAX98357A digital audio amplifier connected via I2S (pins: BCLK → GPIO 19, LRCLK → GPIO 20, DATA → GPIO 21), a speaker, multiple copper touch sensors, push buttons for additional functions, and an LED for status indication. The touch sensors are directly connected to the touch-capable GPIO pins of the ESP32-S3, which measures changes in capacitance to detect finger contact reliably.

For sound generation, a wave table technique is used. A mathematical waveform primarily a sine or sawtooth wave is pre-calculated and stored in an array. By iterating through this array at high speed, audible tones at specific frequencies are produced. Each touch sensor is mapped to a musical note on the scale. When multiple sensors are activated simultaneously, the system generates multiple voices and mixes them together before sending the combined audio signal to the amplifier, enabling chords to be played.

During development, several test stages were carried out. First, a constant 440 Hz test tone verified that the I2S audio output functioned correctly. Then individual touch sensors were tested and threshold values were calibrated using serial monitor output. Finally, polyphony was implemented to allow simultaneous notes. A key challenge was that reading too many touch sensors in quick succession caused processing delays. I was not able to fix the bug yet, the output volume is automatically scaled down when multiple voices are active to prevent audio clipping and distortion.

Conclusion

The project successfully achieved its original goal: a functional digital musical instrument was built from scratch using an ESP32-S3. The synthesizer supports polyphonic playback, real-time touch input, volume control, and octave shifting. Compared to the initial plan, several technical challenges had to be overcome, particularly simultaneous sensor processing and stable audio output. Nevertheless, all core features work reliably. In the future, i will be 3D printing a shell for the electronics to sit, the project could be extended with additional waveforms, audio filters, or even a MIDI interface, which would make the synthesizer significantly more versatile and suitable for more advanced musical applications.